Patterns

Managing Long Running Tasks

Learn about the long running tasks pattern and how to use it in your system design

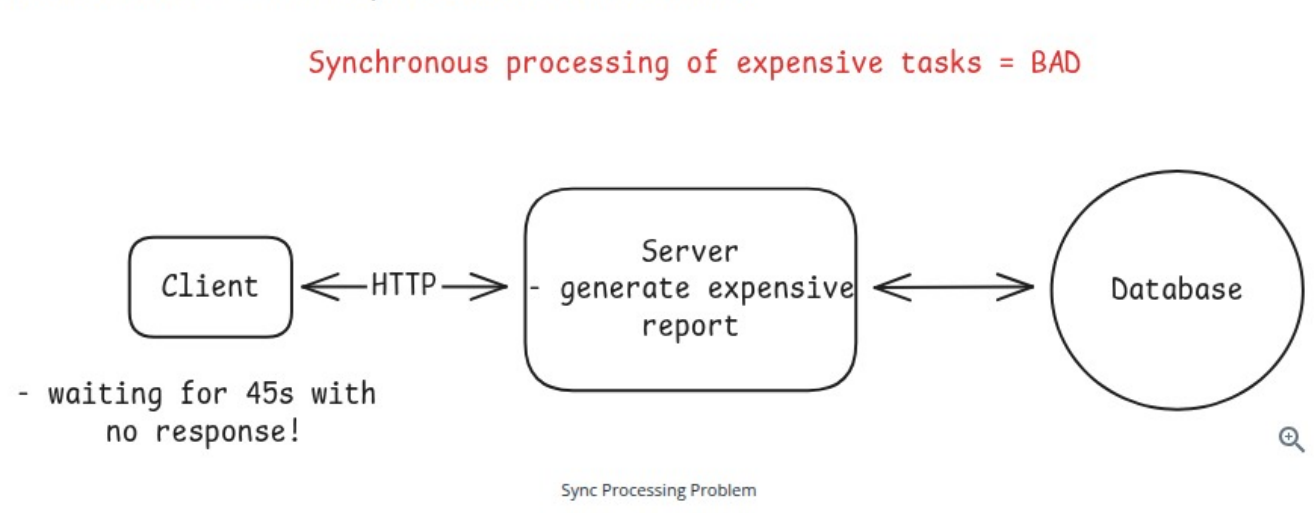
Managing Long Running Tasks

> 🏃 The **Managing Long-Running Tasks** pattern splits API requests into two phases: immediate acknowledgment and background processing. When users submit heavy tasks (like video encoding), the web server instantly validates the request, pushes a job to a queue (Redis/RabbitMQ), and returns a job ID, all within milliseconds. Meanwhile, separate worker processes continuously poll the queue, grab pending jobs, execute the actual time-consuming work, and update the job status in a database.

The Problem

Let's start with a problem. Imagine you run a simple website where users can view their profile. When a user loads their profile page, your server makes a quick database query to fetch their data. The whole process - querying the database, formatting the response, sending it back - takes less than 100 milliseconds. The user clicks and almost instantly sees their information. Life is good.

Now imagine that instead of just fetching data, we need to generate a PDF report of the user's annual activity. This involves querying multiple tables, aggregating data across millions of rows, rendering charts, and producing a formatted document. The whole process takes at least 45 seconds.


Synchronous processing of expensive tasks = BAD
Client
HTTP
Server
- generate expensive report
Database
- waiting for 45s with no response!

Sync Processing Problem

With synchronous processing, the user's browser sits waiting for 45 seconds. Most web servers and load balancers enforce timeout limits around 30-60 seconds, so the request might not even complete. Even if it does, the user experience is poor. They're staring at a loading indicator with no feedback about progress.

The PDF report isn't unique. Video uploads, for example, require transcoding that takes several minutes. Profile photo uploads need resizing, cropping, and generating multiple thumbnail sizes. Bulk operations like sending newsletters to thousands of users or importing large CSV files take even longer. Each of these operations far exceeds what users will reasonably wait for.

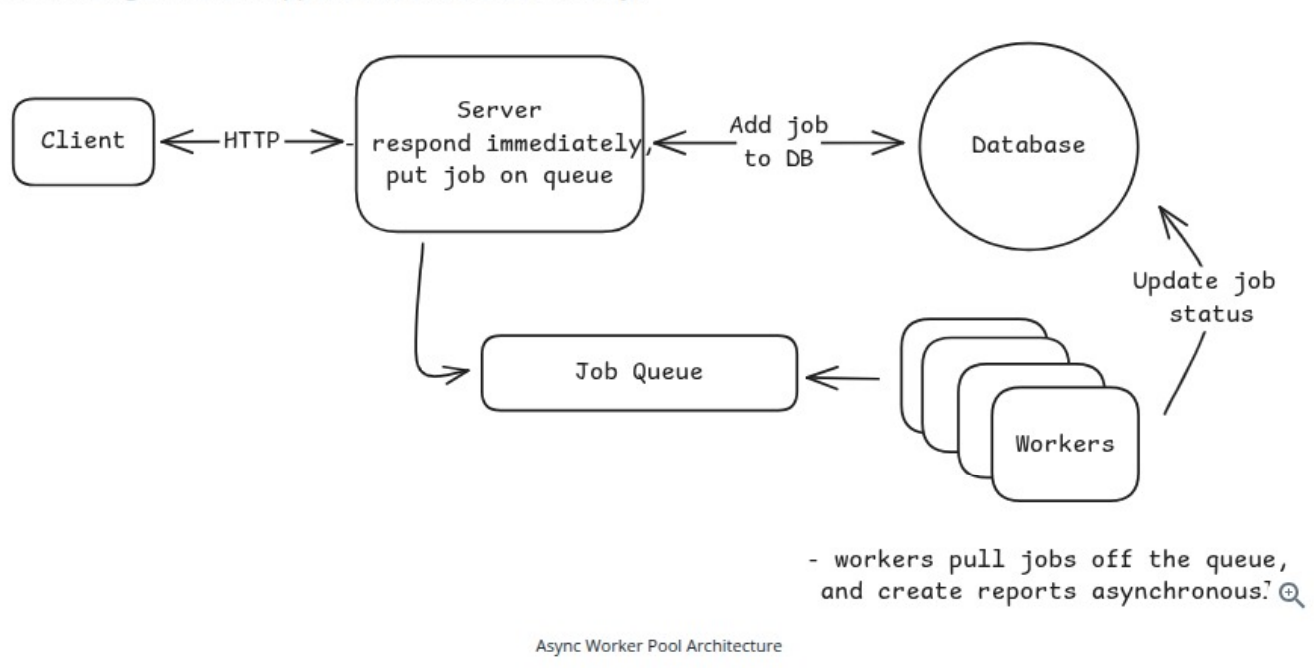
Even if you could scale infinitely, synchronous processing provides terrible user feedback. The user clicks "Generate Report" and then... nothing. Is it working? Did it fail? Should they refresh? After 30 seconds of waiting, many users assume something broke and try again, creating duplicate work that makes the problem worse.

Alright, so synchronous processing clearly has its limits when operations take more than a few seconds. What's the alternative?

The Solution

Instead of making the user wait while we generate their PDF, we split the operation into two parts. When they click "Generate Report", we immediately store their request in a queue and return a response: "We're generating your report. We'll notify you when it's ready." This takes milliseconds, not minutes.

The web server's job is now super simple. It just validates the request, adds it to a queue, and returns a job ID. The actual PDF generation happens somewhere else entirely.

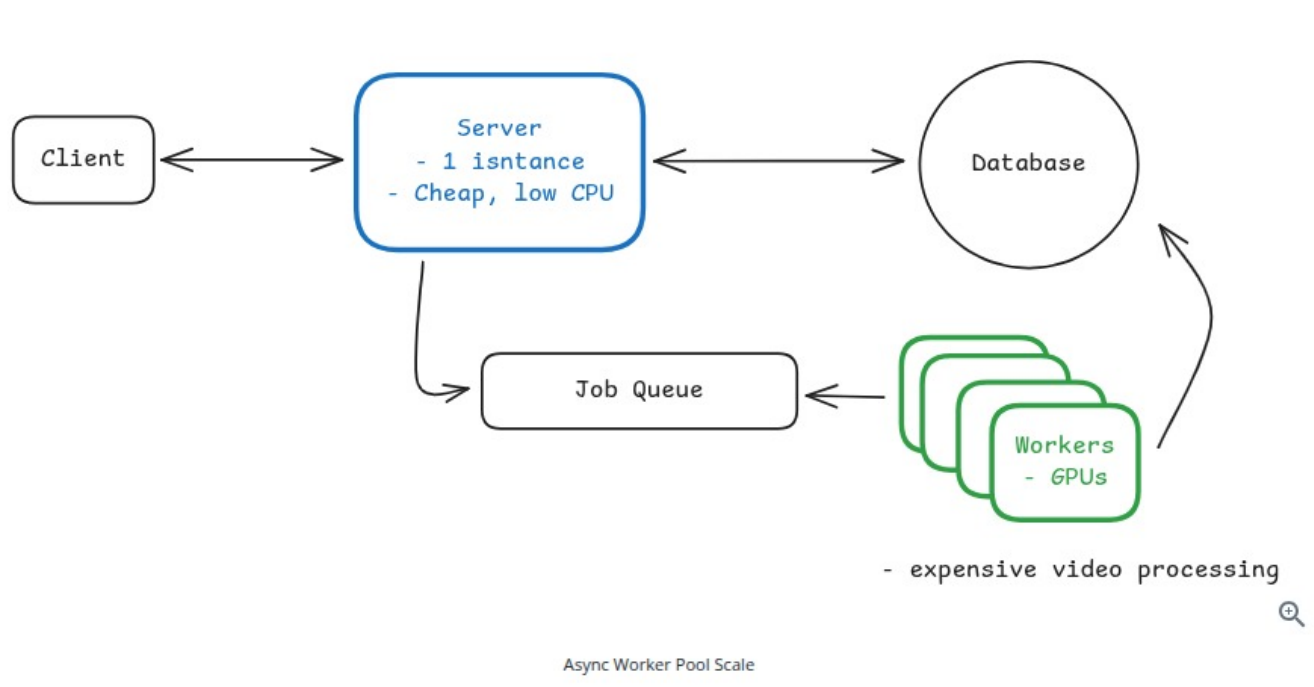


Async Worker Pool Architecture

We call this the **Managing Long-Running Tasks pattern**. The core idea is to decouple request acceptance from request processing. Your web servers become lightweight request routers that quickly acknowledge work and hand it off. A separate pool of worker processes handles the actual heavy lifting. These workers pull jobs from a shared queue, process them at their own pace, and update the job status when complete. The "async" part means the original HTTP request completes immediately without waiting for the work to finish. The "worker pool" part refers to the collection of processes dedicated to executing these background jobs.

The queue acts as a buffer between your web servers and workers. It stores jobs durably so nothing gets lost if a worker crashes. Popular choices for queues include Redis (with Bull or BullMQ), AWS SQS, RabbitMQ, or Kafka for higher scale. The queue also gives you visibility into how many jobs are waiting, how long they're taking, and whether any have failed.

This separation solves the scaling problems we saw earlier. Your web servers no longer need expensive GPUs just because some requests trigger video processing. You can run web servers on small, cheap instances while your video workers use GPU-optimized machines. When PDF generation backs up during month-end reporting, you spin up extra workers without touching your web server fleet. Each component scales based on its specific needs.



Async Worker Pool Scale

The user experience improves dramatically too. Instead of staring at a frozen browser, users get immediate confirmation that their request was received. They can check back later to see their position in the queue or processing progress. When the job completes, you notify them via email, push notification, or WebSocket update. They're free to navigate away, close their laptop, and come back when the work is actually done.

This pattern applies broadly to any operation that takes more than a few seconds. Image processing, video transcoding, bulk data imports, third-party API calls with strict rate limits, report generation, email campaigns - they can all benefit from async processing. The specific queue technology and worker implementation might vary, but the core pattern remains the same. Accept quickly, process asynchronously, notify when complete.

Trade-offs

Managing long-running tasks isn't magic. This approach solves some problems but creates others. Here's what you're getting into:

What you gain

- **Fast user response times** - API calls return in milliseconds instead of timing out after 30 seconds. Users get immediate acknowledgment that their request was received.
- **Independent scaling** - Web servers and workers scale separately. Add more workers during peak processing times without paying for idle web servers.
- **Fault isolation** - A worker crash processing one video doesn't bring down your entire API. Failed jobs can be retried without affecting user-facing services.
- **Better resource utilization** - CPU-intensive workers run on compute-optimized instances. Memory-heavy tasks get high-memory machines. Web servers use cheap, general-purpose instances.

What you lose

- **System complexity** - You now have queues, workers, and job status tracking to manage. More moving parts means more things that can break.
- **Eventual consistency** - The work isn't done when the API returns. Users might see stale data until background processing completes.
- **Job status tracking** - You need infrastructure to store job states, handle retries, and expose status endpoints. This adds database load and API complexity.
- **Monitoring overhead** - Queue depth, worker health, job failure rates, processing latency - you're monitoring a distributed system instead of simple request/response cycles.

Plus async patterns create new failure modes. What happens when the queue fills up? How do you handle poison messages that crash workers repeatedly? When do you give up retrying a failed job? These problems aren't impossible to solve, but they need planning that synchronous systems don't.

How to Implement

At its core, two technologies are required to pull this off:

1. **Message Queue**: A message queue is a system that allows you to send and receive messages between different parts of your system. It's a way to decouple the sender and receiver of a message, allowing them to operate independently.
2. **Pool of Workers**: A pool of workers is a group of processes that are dedicated to processing messages from a queue. They are responsible for pulling messages from the queue, processing them, and updating the job status.

Let's break down each component and the common technology choices you'll discuss in interviews.

Message Queue

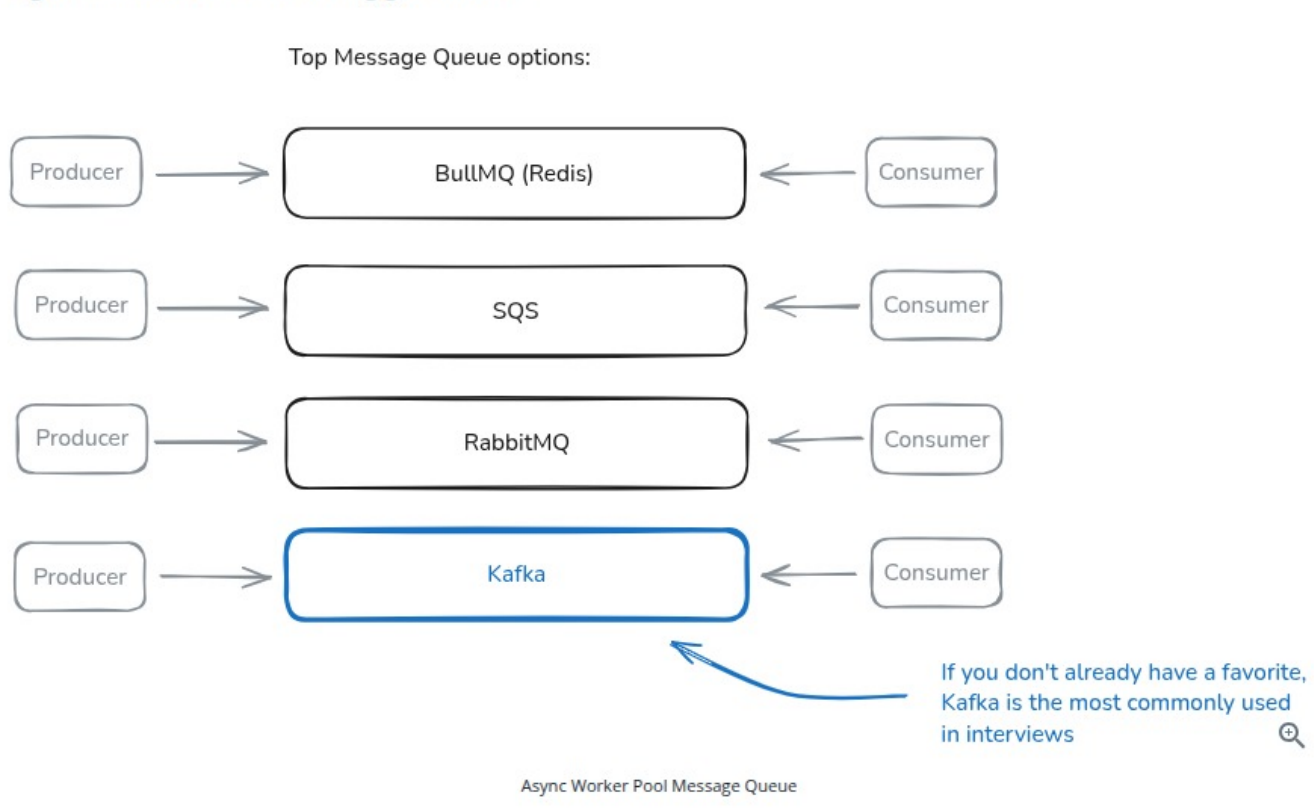
The queue needs to be durable. If it crashes, you can't lose all pending jobs. It also needs to handle concurrent access from multiple workers without duplicating work. Here are the standard options:

Redis with Bull/BullMQ is the go-to for many startups. Redis provides the storage, while Bull adds job queue semantics on top: automatic retries, delayed jobs, priority queues. It's simple to set up and "just works" for 90% of use cases. While Redis offers persistence options, it's still memory-first, so you can lose jobs in a hard crash. For true durability, you'd want SQS or RabbitMQ.

AWS SQS removes the operational overhead entirely which is nice. Amazon manages the infrastructure, handles scaling, and guarantees message delivery. You pay per message, which is great at low volume but can get expensive at scale. The 256KB message size limit means you're storing job data elsewhere and just passing IDs through the queue.

RabbitMQ gives you more control with complex routing patterns, but requires self-hosting. It's battle-tested in enterprise environments and handles sophisticated workflows well. The operational burden is real though. You need to manage clusters, handle upgrades, and monitor disk usage.

Kafka is the go-to, especially when your stack already uses event streaming. Its append-only log lets you replay messages, fan-out to multiple consumers, and keep data around for long retention windows—while still handling huge volumes with strict ordering guarantees.


Top Message Queue options:
Producer
BullMQ (Redis)
Consumer
Producer
SQS
Consumer
Producer
RabbitMQ
Consumer
Producer
Kafka
Consumer
If you don't already have a favorite, Kafka is the most commonly used in interviews

Async Worker Pool Message Queue

Workers

Workers pull jobs from the queue and execute them. The key decision is how to run these workers:

"Normal" servers are the simplest approach. You run 10-20 worker processes on a few machines, each pulling jobs in a loop. This gives you full control over the environment and makes debugging straightforward since you can SSH into the box and inspect what's happening. Long-running jobs aren't a problem when you control the infrastructure. The downside is you're managing servers and paying for idle capacity during quiet periods.

```
# Simplified worker loop
while True:
    job = queue.pop()  # Blocks until job available
    if job:
        process_job(job)
        mark_complete(job.id)
```

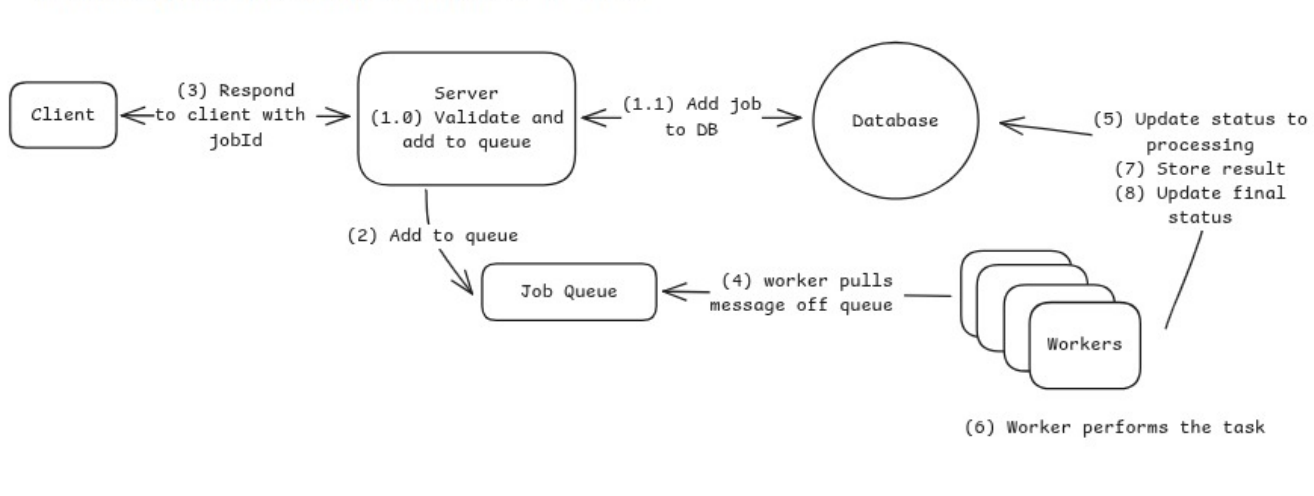
Serverless functions (Lambda, Cloud Functions) eliminate server management. Each job triggers a function execution that scales automatically. This works beautifully for spiky workloads where you might process 1,000 jobs one minute and zero the next. You only pay for actual execution time. But the constraints are meaningful here. You're limited to 15-minute executions, cold starts add latency, and local storage is minimal.

Container-based workers (on Kubernetes or ECS) offer a middle ground. You package workers as Docker containers and let the orchestrator handle scaling and deployment. This approach is more complex than plain servers but gives you more flexibility than serverless. You can still handle long-running jobs while getting better resource utilization through automatic scaling.

Putting It Together

Here is how everything works together:

1. Web server validates the request and creates a job record in your database with status "pending"
2. Web server pushes a message to the queue containing the job ID (not the full job data)
3. Web server returns the job ID to the client immediately
4. Worker pulls message from queue, fetches job details from database
5. Worker updates job status to "processing"
6. Worker performs the actual work
7. Worker stores results (in S3 for files, database for metadata, etc.)
8. Worker updates job status to "completed" or "failed"


Client
(3) Respond to client with jobId
Server
(1.0) Validate and add to queue
(1.1) Add job to DB
Database
(5) Update status to processing
(7) Store result
(8) Update final status
(2) Add to queue
Job Queue
(4) worker pulls message off queue
Workers
(6) Worker performs the task

Async Worker Pool Putting It Together

Importantly, each component can fail independently and the system will still function. If the queue goes down, web servers still accept requests and jobs pile up in the database as "pending". When workers crash, jobs stay in the queue until new workers come online. If the database has issues, workers can retry until it recovers.

> ⚠ In interviews, don't overcomplicate this. Pick Kafka (or a queue you know well) for the queue unless there's a specific reason not to. Use regular server processes for workers unless the interviewer pushes for serverless. Focus on showing that you understand the separation of concerns rather than debating the merits of different queue technologies.

When to Use in Interviews

Don't wait for the interviewer to ask about managing long-running tasks. The key is recognizing problems that scream for async processing and proactively suggesting it.

Here are some common signals to watch for that should trigger your thinking about managing long-running tasks:

1. **When they mention specific slow operations** - The moment you hear "video transcoding", "image processing", "PDF generation", "sending bulk emails", or "data exports" that's your cue. These operations take seconds to minutes. Jump in immediately: "Video transcoding will take several minutes, so I'll return a job ID right away and process it asynchronously."

2. **When the math doesn't work** - If they say "we process 1 million images per day" and you know image processing takes 10 seconds, do the quick calculation out loud: "That's about 12 images per second, which means 120 seconds of processing time per second. We'd need 120+ servers just for image processing. I'll use async workers instead."

3. **When different operations need different hardware** - If the problem involves both simple API requests and GPU-heavy work (like ML inference or video processing), that's a clear async signal. "We shouldn't run GPU workloads on the same servers handling login requests. I'll separate these with async workers on GPU instances."

4. **When they ask about scale or failures** - Questions like "what if a server crashes during processing?" or "how do you handle 10x traffic?" are perfect openings to introduce async workers. "With async workers, if one crashes mid-job, another worker picks it up from the queue. No user requests are lost."

> 💡 The skill is being proactive. You should be the one pointing out that operations will take too long, not waiting for them to ask. This shows system design maturity, you're thinking about timeouts, resource utilization, and user experience before they even bring up these concerns.

For Example

Let's look at how managing long-running tasks shows up in popular system design interview questions. For each of these, we have detailed breakdowns showing exactly how async workers fit into the broader system design:

YouTube/Video Platforms - Video upload triggers multiple async jobs: transcoding to different resolutions (1080p, 720p, 480p), generating thumbnails, extracting closed captions, running content moderation. Each video might need hours of processing across multiple workers.

Instagram/Photo Sharing - Photo uploads spawn async tasks for generating multiple sizes, applying filters, extracting metadata, running image recognition for tags, and checking content policy. The fan-out problem also uses async workers: when someone with millions of followers posts, workers propagate that post to follower feeds.

Uber/Ridesharing - Ride matching runs asynchronously. The user sees "Finding drivers near you..." while workers evaluate driver availability, calculate optimal routes, and determine pricing. Driver location updates also process through async workers to update rider apps without overloading the main API.

Stripe/Payment Processing - Payment APIs return immediately with a pending status. Async workers handle the actual charge attempt, fraud detection, 3D Secure verification, and webhook notifications. This isolation prevents slow bank APIs from blocking the merchant's checkout flow.

Dropbox/File Sync - File uploads trigger async processing: virus scanning, indexing for search, generating previews, and syncing to other devices. Large files might take minutes to process, but the upload API returns immediately.

Notice the pattern: any operation involving heavy computation, external API calls, or fan-out to multiple users benefits from async processing. If the interviewer mentions any of these scenarios, managing long-running tasks should be part of your solution.

Common Deep Dives

As you already know, interviewers love to catch you off guard and need to dig. Here are the five common areas of exploration, along with how to handle each one. It may be that you bring these up proactively, or they may be asked of you.

Handling Failures

Your interviewer might start with a basic question like "what happens if the worker crashes while working the job?" The answer is pretty simple if we've been following the pattern. The job will be retried by another worker.

But how do we know the worker crashed? Typically, you'll have a heartbeat mechanism that periodically checks in with the queue to let it know that the worker is still alive. If the worker doesn't check in, the queue will assume it's dead and will retry the job.

The interval of the heartbeat is a key design decision. If it's too long, crashes will mean the jobs are significantly delayed (the queue will optimistically assume the worker is still alive for much longer than it actually is). If it's too short, you'll be sending a lot of unnecessary messages to the queue or worse, you may mark jobs as failed when in fact they're still running (e.g. a garbage collection pause).

Each queue system has a different way of handling this. For example, in SQS, you can set a visibility timeout. In RabbitMQ, you can set a heartbeat interval. In Kafka, you can set a session timeout. Choose the longest interval that is permissible by your users/clients in terms of extra delay. For most systems, 10-30 seconds is a good starting point.

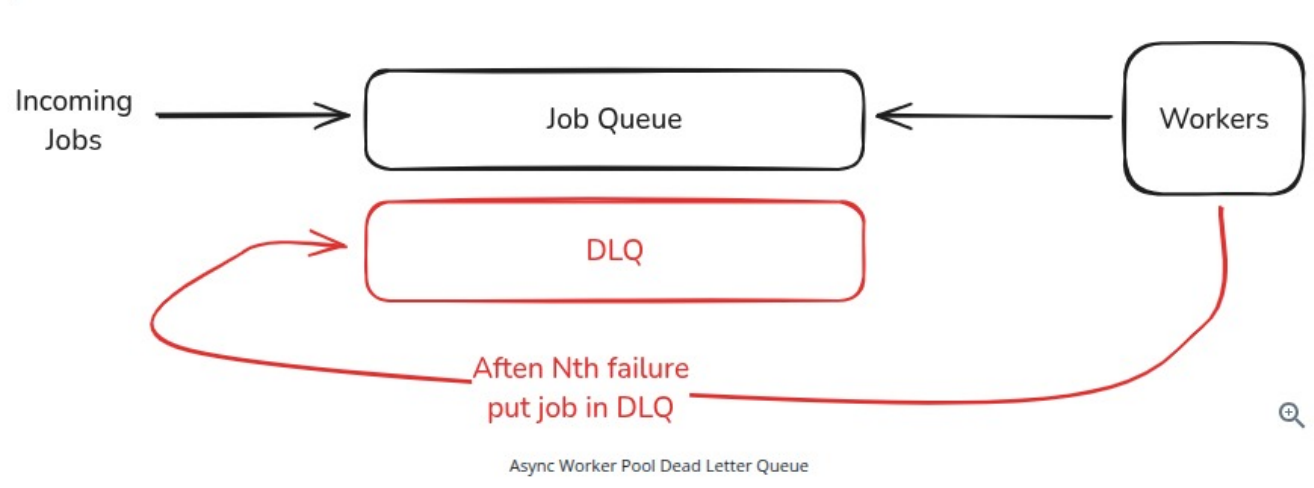
Handling Repeated Failures

The interviewer asks: "What happens if a job keeps failing? Maybe there's a bug in your code or bad input data that crashes the worker every time."

Without handling, failed jobs get retried forever. Your workers waste cycles on doomed jobs while valid work backs up in the queue. Even worse, a poison message that crashes workers can take down your entire worker fleet as each one tries to process it and dies.

The solution is a Dead Letter Queue (DLQ). After a job fails a certain number of times (typically 3-5), you move it to a separate queue instead of retrying again. This isolates problematic jobs while letting healthy work continue. Your DLQ becomes a collection of jobs that need human investigation. Maybe there's a bug to fix or data to clean up. Once fixed, you can move jobs back to the main queue for reprocessing.

Most queue systems have built-in DLQ support. In SQS, you configure a redrive policy. In RabbitMQ, you set up a dead letter exchange. The key is monitoring your DLQ since a growing dead letter queue usually signals a bug in your system that needs immediate attention.



Async Worker Pool Dead Letter Queue

Preventing Duplicate Work

The interviewer continues: "A user gets impatient and clicks 'Generate Report' three times. Now you have three identical jobs in the queue. How do you prevent doing the same work multiple times?"

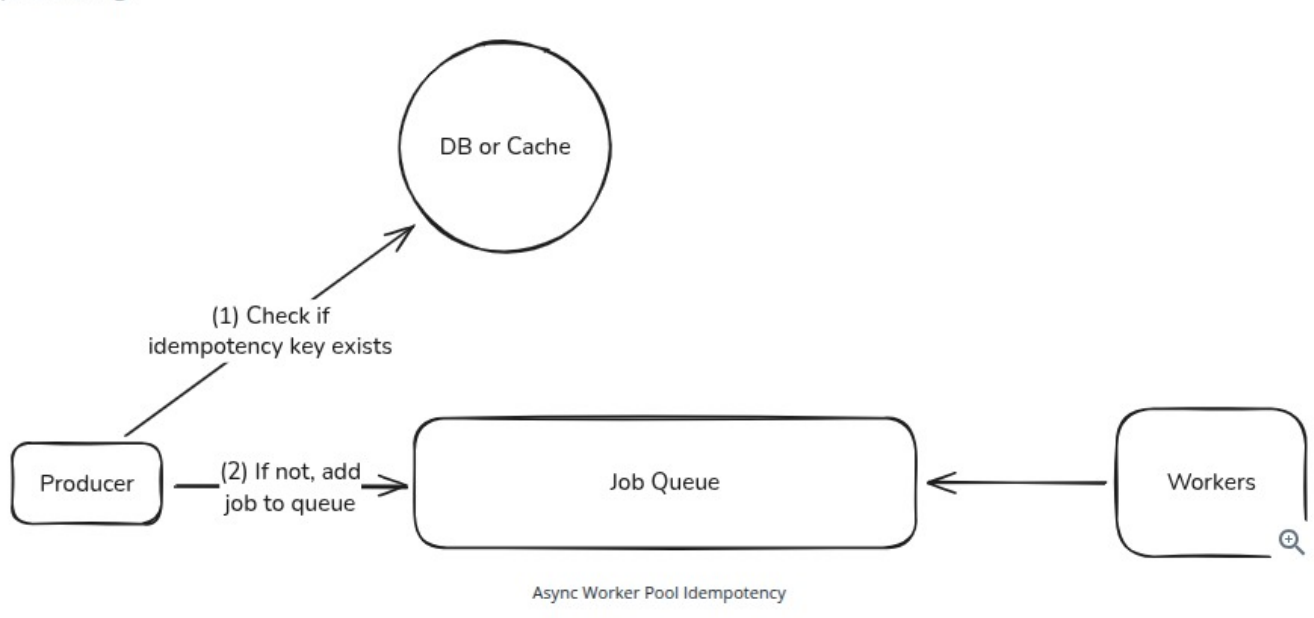
Without deduplication, you waste resources processing identical jobs. Worse, you might send three emails to the same user or charge their credit card three times. Side effects compound when jobs aren't idempotent.

The solution is idempotency keys. When accepting a job, require a unique identifier that represents the operation. For user-initiated actions, combine user ID + action + timestamp (likely rounded to the duration you want to prevent duplicate work on). For system-generated jobs, use deterministic IDs based on the input data. Before starting work, check if a job with this key already exists. If it does, return the existing job ID instead of creating a new one.

```
def submit_job(user_id, job_type, job_data, idempotency_key):
    # Check if job already exists
    existing_job = db.get_job_by_key(idempotency_key)
    if existing_job:
        return existing_job.id

    # Create new job
    job_id = create_job(user_id, job_type, job_data)
    db.store_idempotency_key(idempotency_key, job_id)
    queue.push(job_id)
    return job_id
```

You'll also want to make the work itself idempotent. If a job fails halfway through and gets retried, it should be safe to run again. This might mean checking if an email was already sent or if a file was already processed before proceeding.


DB or Cache
(1) Check if idempotency key exists
Producer
(2) If not, add job to queue
Job Queue
Workers

Async Worker Pool Idempotency

Managing Queue Backpressure

Next question: "It's Black Friday and suddenly you're getting 10x more jobs than usual. Your workers can't keep up. The queue grows to millions of pending jobs. What do you do?"

When workers can't process jobs fast enough, queues grow unbounded. Memory usage explodes. Job wait times stretch to hours. New jobs get rejected because the queue is full. Users get frustrated waiting for results that never come.

The solution is called backpressure. It slows down job acceptance when workers are overwhelmed. You can set queue depth limits and reject new jobs when the queue is too deep and return a "system busy" response immediately rather than accepting work you can't handle.

You should also autoscale workers based on queue depth. When the queue grows beyond a threshold, spin up more workers. When it shrinks, scale down. CloudWatch alarms + Auto Scaling groups make this straightforward on AWS. The key metric is queue depth, not CPU usage. By the time CPU is high, your queue is already backed up.

Handling Mixed Workloads

The interviewer probes: "Some of your PDF reports take 5 seconds, but end-of-year reports take 5 hours. They're all in the same queue. What problems does this cause?"

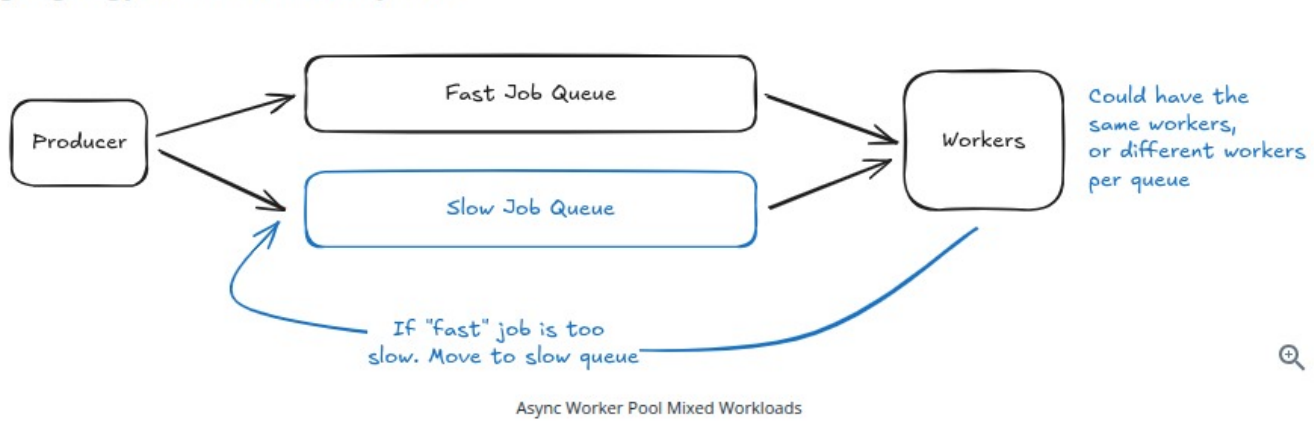
Long jobs block short ones. A user requesting a simple report waits hours behind someone's massive year-end export. Worker utilization becomes uneven as some workers process hundreds of quick jobs while others are stuck on single long jobs. Autoscaling becomes a huge pain when you can't predict how long jobs will take.

The solution is to separate queues by job type or expected duration. Quick reports go to a "fast" queue with many workers. Complex reports go to a "slow" queue with fewer, beefier workers. This prevents head-of-line blocking and lets you tune each queue independently. Alternatively, you can break large jobs into smaller chunks that use the same queue infrastructure, like splitting news feed fanout into batches of followers.

```
queues:
  fast:
    max_duration: 60s
    worker_count: 50
    instance_type: t3.medium

  slow:
    max_duration: 6h
    worker_count: 10
    instance_type: c5.xlarge
```

Route jobs to the appropriate queue when submitted. If you can't predict duration upfront, start jobs in the fast queue and move them to the slow queue if they exceed a time limit. This keeps your fast queue responsive while giving long jobs the resources they need.


Producer
Fast Job Queue
Slow Job Queue
Workers
Could have the same workers, or different workers per queue
If "fast" job is too slow. Move to slow queue

Async Worker Pool Mixed Workloads

Orchestrating Job Dependencies

Finally: "What if generating a report requires three steps: fetch data, generate PDF, then email it. How do you handle jobs that depend on other jobs?"

Without proper orchestration, you end up with spaghetti code. Workers complete step 1 then directly queue step 2, making the flow hard to understand and modify. Error handling becomes a nightmare - if step 2 fails, should you retry just that step or start over? Monitoring which step failed and why requires digging through logs.

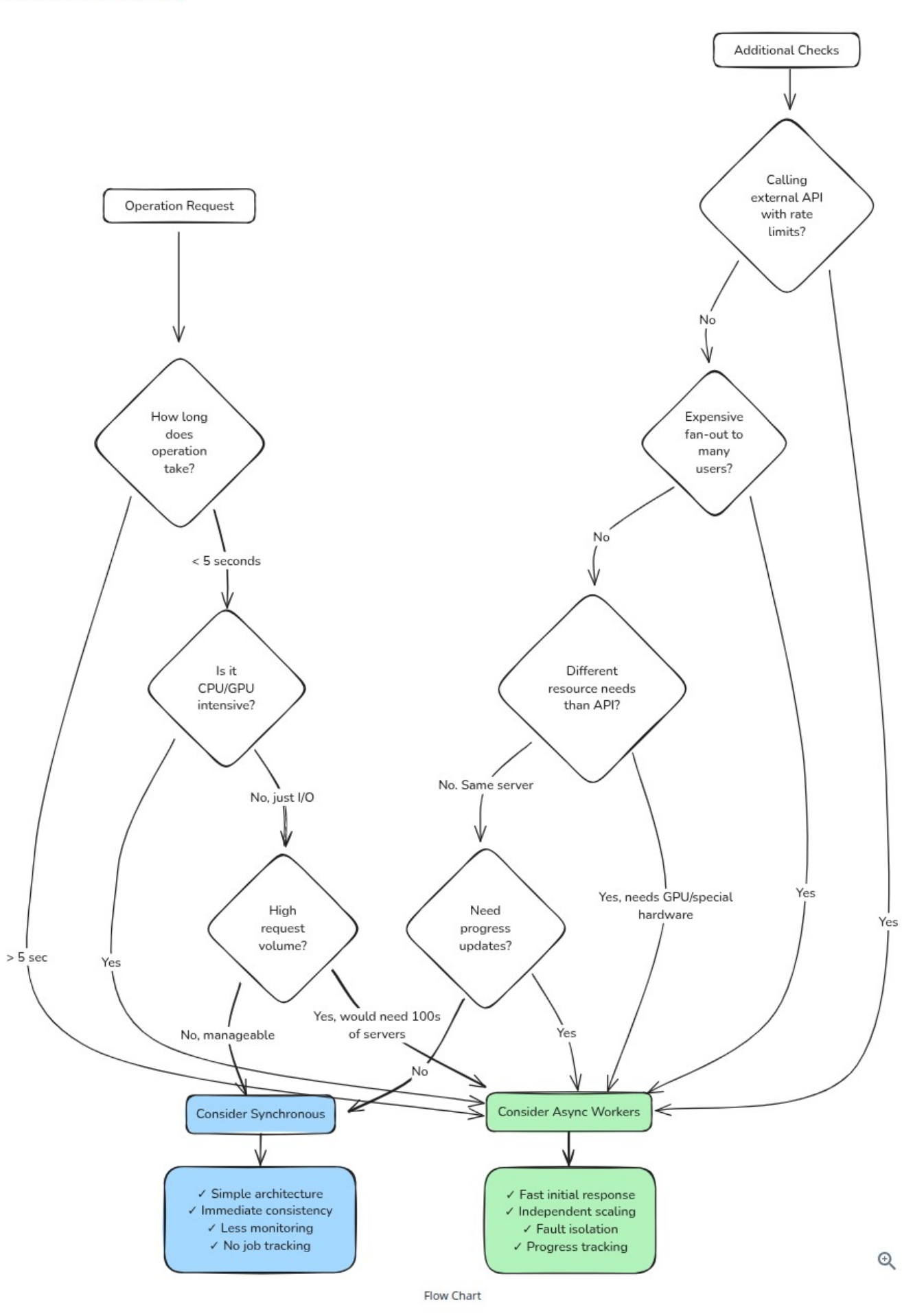
For simple chains, have each worker queue the next job before marking itself complete. Include the full context in each job so steps can be retried independently:

```
{
  "workflow_id": "report_123",
  "step": "generate_pdf",
  "previous_steps": ["fetch_data"],
  "context": {
    "user_id": 456,
    "data_s3_url": "s3://bucket/data.json"
  }
}
```

For complex workflows with branching or parallel steps, use a workflow orchestrator like **AWS Step Functions**, **Temporal**, or **Airflow**. These tools let you define workflows as code, handle retries per step, and provide visibility into where workflows get stuck. The tradeoff is additional complexity - only reach for orchestration when job dependencies become truly complex.

More information on this can be found in the **Multi-Step Processes** pattern.

Conclusion



Flow Chart

The Managing Long-Running Tasks pattern is one that shows up a lot in system design interviews. Once you recognize the signs - operations taking more than a few seconds, resource mismatches, or scale requirements that don't add up - the solution becomes obvious. Queue the work, return immediately, process asynchronously.

Remember that in interviews, you want to be the one identifying these problems, not waiting for the interviewer to point them out. When they mention video uploads or PDF generation, immediately say "that's going to take too long for a synchronous request, so I'll use async workers." Show that you're thinking about timeouts, resource utilization, and user experience from the start.

The implementation details matter less than demonstrating you understand the pattern. Pick a queue technology you're comfortable discussing (Kafka is a safe default), mention the tradeoffs around durability and complexity, and be ready to handle the common deep dives around failure handling and scaling.

Most importantly, don't overthink it. This pattern shows up in nearly every company at scale - from Uber's ride matching to Instagram's feed fanout. Master this pattern and you'll handle a huge class of system design problems with confidence.

Test Your Knowledge
Take a quick 15 question quiz to test what you've learned.

Start Quiz

Mark as read

Add a comment...

Comment

Anonymous

Posting as leetcode

Search comments

Sort By Old

HeadBrownGayal101 • 1 month ago • edited 1 month ago
Can you please add a way for us to add notes and/or highlight the content?
24 Reply

Evan King • 1 month ago
Are you thinking Medium style or have something else in mind?
1 Reply

HeadBrownGayal101 • 1 month ago
Anything's fine as long as I have a way to keep notes so that I can come back to it and quickly revise before the interview.
13 Reply

InvisiblePinkChipmunk238 • 1 month ago
@Evan Suggestion for highlight & notes: Highlight any sentences and as next step add comment - Google doc style would be awesome.
0 Reply

FullTealTrout477 • 25 days ago
there are text highlighting chrome extensions
0 Reply

ChristianPlumTakin500 • 1 month ago
@Evan King, Notes would be super helpful! These articles are great refreshers to go through before interviews, rather than going through a lot of questions. I'm making notes separately based on these articles, but would love to have

1. Notes per article
2. List of all my notes across the site
3. Notes not tied to any article

This would allow users to stay within your website. And you can further use AI to analyze these notes as feedback loop to improve the posts

```
Note
- note_id
- user_id
- post_id
- content_s3_url
- created_at
- updated_at
```

Show More

9 Reply

CooperativeRedAntelope278 • 1 month ago • edited 1 month ago
Love the new patterns you've recently added. Thank you so much for all the hard work.
15 Reply

Evan King • 1 month ago
Thrilled you like them! More on the way soon
15 Reply

CasualAquamarineRhinoceros625 • 1 month ago
Can't wait!
1 Reply

Sujeet Kumar • 1 month ago • edited 1 month ago
Thank you for adding this amazing content.
Could you please also elaborate on the notification flow (email, push notification, or WebSocket) after task completion?
0 Reply

Evan King • 1 month ago • edited 1 month ago
This might have what you're looking for? Real-time updates
3 Reply

teja.surya6 • 1 month ago • edited 1 month ago
Thank you so much for the fantastic writeup about system design patterns. What does a poison message when dealing with errors in the message queue mean? Could you please give me an example?
1 Reply

Evan King • 1 month ago
A poison message is a message that causes your worker to crash every time it tries to process it. Example would be like malformed JSON that breaks your parser, or data that triggers an edge case bug in your code. The danger is that this message can take down your entire worker pool as each worker tries to process it, crashes, and then the message gets requeued for the next worker. That's why you need a DLQ to move the poison message to
4 Reply

CasualAquamarineRhinoceros625 • 1 month ago • edited 1 month ago
It would be great to get your take on when to choose Step Functions vs Airflow vs Temporal for workflow orchestration (even if they are mostly identical from a system design interview perspective), obviously I can ask chatgpt but would appreciate your take more than chatgpt's take :)
1 Reply

Stefan Mai • 1 month ago
We cover this in Multi-Step Processes. Prefer Temporal as it's more full-featured and open source unless you're doing ETL work where airflow is appropriate.
3 Reply

CasualAquamarineRhinoceros625 • 1 month ago • edited 1 month ago
Stefan - Thank you, I was a little too excited about your release of these pattern write ups and started reading this write up first and didn't look at all of them yet, I'll take a look, thank you.

At my current job, we use Airflow for data pipelines and Step Functions for non data pipeline use cases.
3 Reply

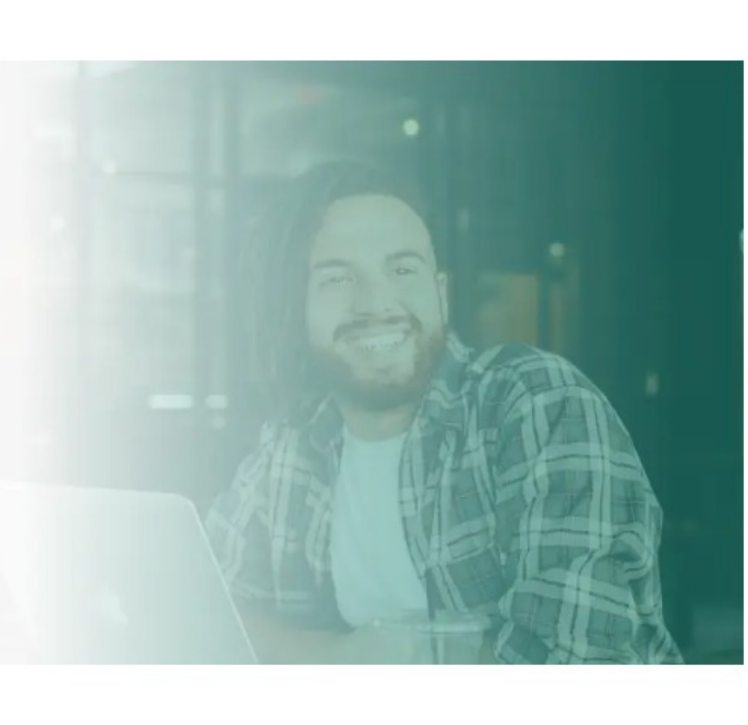
Stefan Mai • 1 month ago
Totally valid choice!
0 Reply

Show All Comments

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

Schedule A Mock Interview



Questions
Meta SWE Interview Questions
Amazon SWE Interview Questions
Google SWE Interview Questions
Engineering Manager (EM) Interview Questions

Learn
Learn System Design
Learn DSA
Learn Behavioral
Learn ML System Design
Guided Practice

Links
FAQ
Pricing
Gift Mock Interviews
Gift Premium
Become a Coach
Our Coaches
Employers
Hello Interview Premium

Legal
Terms and Conditions
Privacy Policy

Contact
About Us
support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.